School of Future Tech

Case Study Report

on

Case Study 94: Secure Online Voting System

by

Pratik Kumar Swain

150096725184

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

In any democratic setup, the integrity of the voting process is paramount. Traditional paper-based systems, while familiar, are often slow, prone to manual counting errors, and difficult to manage as the number of participants grows. Digital transformation offers a way to streamline this process, ensuring that every vote is accounted for quickly and accurately.

This case study focuses on the development of a **Digital Voting System** built using C++ and Object-Oriented Programming (OOP) principles. The goal is to create a secure, console-based environment where voters can register, view candidates, and cast their ballots without the risk of double-voting. By leveraging classes and data structures, the system provides a structured approach to managing sensitive election data.

2. Problem Statement / Case Background (Abstract)

Background

Manual voting systems face several bottlenecks: long wait times, high resource consumption, and potential human error during the tallying phase. Furthermore, ensuring that a voter does not cast multiple ballots requires rigorous manual verification, which is often inefficient. A software-based solution can automate these checks, providing instant results and a clear audit trail of registered voters.

Abstract

This project implements a secure, localized voting application. The system allows an administrator to register voters with unique IDs and passwords and add candidates representing different parties. Using a modular approach, the program handles the election lifecycle—from starting the session to stopping it and declaring a winner based on real-time vote counts. Key features include credential verification, "has-voted" status tracking to prevent fraud, and automated result calculation.

3. Case Study Design

The Digital Voting System is designed as an interactive management tool with a focus on data integrity. The logic is divided into distinct phases:

1. **Identity Management**: Defining the attributes for both `Voter` and `Candidate` (IDs, names, and authentication details).
2. **Election Lifecycle Control**: Functions to toggle the `isRunning` state, ensuring votes are only cast when the election is officially "live."

3. **Authentication & Validation**: A multi-step verification process where the system checks the voter's ID against their password and confirms their eligibility (i.e., whether they have already voted).

4. **Data Storage**: Using dynamic vectors to store a growing list of participants and candidates during the program's execution.

5. **Result Aggregation**: A logic-based comparison to determine the winner by iterating through candidate scores and identifying the highest tally.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Key Methods and Concepts:

- **Object-Oriented Programming (OOP):** The system is built around classes (`Voter`, `Candidate`, `Election`), encapsulating data and behavior.
- **Encapsulation:** Grouping related data (like `voteCount` and `party`) within the `Candidate` class.
- **Linear Search Algorithm:** Used to verify voter credentials and find candidate IDs within the vectors.
- **Flag-based Logic:** Utilizing boolean variables (like `hasVoted` and `isRunning`) to enforce business rules and prevent illegal actions.

Technology Stack:

- **Programming Language:** C++.
- **Standard Template Library (STL):** Specifically `std::vector` for dynamic memory management.
- **Input/Output:** `iostream` for handling console interactions.
- **Environment:** Any standard C++ compiler (GCC, Clang, or MSVC) and IDEs like VS Code or Code::Blocks.

5. Problem Statement / Case Study Implementation Details and Snapshots.

The implementation is organized into a modular C++ class structure, ensuring that data related to voters and candidates is encapsulated and protected. Below are the key components and visual representations of the system in action:

A. Modular Code Structure

- **Entity Classes**: The `Voter` and `Candidate` classes store essential attributes like unique IDs, names, and passwords.
- **Election Management**: The `Election` class contains the core logic for adding participants, managing the election state, and calculating results.
- **Menu-Driven Interface**: The `main()` function implements a continuous loop that allows users to navigate through various system options seamlessly.

B. System Snapshots

A. Voter Registration and Data Entry

The first step involves populating the system with eligible voters. The code uses a `Voter` class to store credentials.

```
// From casstudy.cpp: Logic for adding a new voter

void addVoter(){

    Voter v;

    cout << "Voter ID: "; cin >> v.id;

    cout << "Name: "; cin >> v.name;

    cout << "Password: "; cin >> v.pass;

    v.hasVoted = false;

    voters.push_back(v);

}
```

B. Secure Voting Logic

The `castVote()` function is the core of the system. It performs three critical checks: verifying the election is live, authenticating the user, and ensuring the user hasn't already voted.

C++

```
// From casstudy.cpp: Security checks before allowing a vote

if(voters[i].id == uid && voters[i].pass == pwd){

    idx = i; // Authentication successful

}
```

```
...

if(voters[idx].hasVoted){

    cout << "You've already voted!\n";

    return;

}
```

C. Real-time Results and Winner Determination

Once the election is stopped using the `stopElection()` method, the results are calculated by iterating through the `cands` vector to find the maximum `voteCount`.

C++

```
// From casstudy.cpp: Logic to determine the winner

for(int i=0; i<cands.size(); i++){

    if(cands[i].voteCount > maxV){

        maxV = cands[i].voteCount;

        winnerIdx = i;

    }

}
```

Code Output:-

```
==== Voting System ====
1. Register voter
2. Add candidate
3. Start election
4. Vote
5. Stop election
6. Results
7. View voters
8. Exit
Choice: 1
Voter ID: 12345
Name: Pratik
Password: Atp
Done. Voter added.

==== Voting System ====
1. Register voter
2. Add candidate
3. Start election
4. Vote
5. Stop election
6. Results
7. View voters
8. Exit
Choice: 2
Candidate ID: 123
Name: Modi
Party: BJP
Candidate added

==== Voting System ====
1. Register voter
2. Add candidate
3. Start election
4. Vote
5. Stop election
6. Results
7. View voters
8. Exit
Choice: 3
Election is now live

==== Voting System ====
1. Register voter
2. Add candidate
3. Start election
4. Vote
5. Stop election
6. Results
```

```
==== Voting System ====
1. Register voter
2. Add candidate
3. Start election
4. Vote
5. Stop election
6. Results
7. View voters
8. Exit
Choice: 4
Your voter ID: 12345
Password: Atp

-- Candidates --
123. Modi  [BJP]
Enter candidate ID to vote: 123
Confirm? (y/n): y
Vote recorded!

==== Voting System ====
1. Register voter
2. Add candidate
3. Start election
4. Vote
5. Stop election
6. Results
7. View voters
8. Exit
Choice: 5
Election stopped

==== Voting System ====
1. Register voter
2. Add candidate
3. Start election
4. Vote
5. Stop election
6. Results
7. View voters
8. Exit
Choice: 6

===== Results =====
Modi (BJP) — 1 votes

Winner: Modi with 1 votes

==== Voting System ====
1. Register voter
2. Add candidate
3. Start election
4. Vote
```

```
Choice: 6

===== Results =====
Modi (BJP) - 1 votes

Winner: Modi with 1 votes

==== Voting System ====
1. Register voter
2. Add candidate
3. Start election
4. Vote
5. Stop election
6. Results
7. View voters
8. Exit
Choice: 7

All voters:
12345 - Pratik (voted)

==== Voting System ====
1. Register voter
2. Add candidate
3. Start election
4. Vote
5. Stop election
6. Results
7. View voters
8. Exit
Choice: 8
Bye
```

6. Problem Statement / Case Study Results and Conclusion.

Findings

- **Security through Logic: By implementing a hasVoted flag within the Voter object, the system effectively eliminates the possibility of double-voting, which is a major concern in manual systems.**
- **State Control: The isRunning boolean ensures that no votes can be cast before the election officially starts or after it has been stopped by an official.**
- **Efficiency: Digital tallying provides instant results, removing the need for manual counting and reducing the margin of error to zero.**

Conclusion

This case study demonstrates a functional and reliable Digital Voting System built with C++. By integrating OOP principles such as encapsulation and using structured logic for authentication, the project provides a secure foundation for electronic voting. The modular design allows for future enhancements, such as database integration or encrypted password storage, to further improve the system's robustness.

7. References

- C++ Standard Library Documentation for `std::vector` and `std::string` management.
- Object-Oriented Programming (OOP) design patterns for entity relationship management.
- Technical tutorials on building console-based management systems in C++.